

Nome: Guilherme de Moraes China Costa

Data: 09/12/2025

Título: Engenharia de Testes de Software - AT

Link da implementação:

https://github.com/guilhermecosta/INFNET/tree/master/engenharia_de_teste_software/AT

Especificação Funcional - Calculadora de IMC

1. Visão Geral

Sistema para cálculo e classificação do Índice de Massa Corporal (IMC) de acordo com os padrões da Organização Mundial da Saúde (OMS).

2. Requisitos Funcionais

RF01 - Cálculo de IMC

Descrição: O sistema deve calcular o IMC usando a fórmula: $IMC = peso / (altura^2)$

Entradas:

- Peso (kg): número decimal > 0 e ≤ 500
- Altura (m): número decimal ≥ 0.5 e ≤ 3.0

Saída: Valor numérico do IMC com precisão de 2 casas decimais

Regras de Negócio:

- Peso deve ser maior que zero
- Altura deve ser maior que zero
- Peso não pode exceder 500kg (validação de realidade)
- Altura deve estar entre 0.5m e 3.0m (validação de realidade)

RF02 - Classificação do IMC

Descrição: O sistema deve classificar o IMC calculado de acordo com a tabela estabelecida

Faixa de IMC e Classificação

- IMC < 16.0
Classificação: Magreza grave
- IMC entre 16.0 e 16.9
Classificação: Magreza moderada

- IMC entre 17.0 e 18.4
Classificação: Magreza leve
- IMC entre 18.5 e 24.9
Classificação: Saudável
- IMC entre 25.0 e 29.9
Classificação: Sobrepeso
- IMC entre 30.0 e 34.9
Classificação: Obesidade Grau I
- IMC entre 35.0 e 39.9
Classificação: Obesidade Grau II
- IMC ≥ 40.0
Classificação: Obesidade Grau III

RF03 - Validação de Entradas

Descrição: O sistema deve validar todas as entradas antes do cálculo

Validações:

- Rejeitar valores nulos
- Rejeitar valores negativos
- Rejeitar valores fora dos limites estabelecidos
- Apresentar mensagens de erro claras e específicas

3. Requisitos Não Funcionais

RNF01 - Precisão

- Cálculos devem ter precisão de pelo menos 2 casas decimais

RNF02 - Performance

- Cálculo deve ser executado em menos de 100ms

RNF03 - Usabilidade

- Mensagens de erro devem ser claras e orientar o usuário

4. Casos de Teste Principais

Partições de Equivalência - Peso

1. **Válidas:** 0.1 a 500 kg
2. **Inválidas:** ≤ 0 , > 500

Partições de Equivalência - Altura

1. **Válidas:** 0.5 a 3.0 m
2. **Inválidas:** < 0.5 , > 3.0 , ≤ 0

Valores Limite - Peso

- Mínimo válido: 0.1 kg
- Máximo válido: 500 kg
- Valores limítrofes: 0, 0.1, 499.9, 500, 500.1

Valores Limite - Altura

- Mínima válida: 0.5 m
- Máxima válida: 3.0 m
- Valores limítrofes: 0, 0.49, 0.5, 3.0, 3.01

Valores Limite - Classificação

- Teste nos limites de cada faixa: 18.4, 18.5, 24.9, 25.0, etc.

5. Cenários de Teste Exploratório

Cenário 1: Entrada Típica

- **Dados:** Peso 70kg, Altura 1.75m
- **Resultado Esperado:** IMC 22.86, Classificação "Peso normal"

Cenário 2: Valores Extremos Válidos

- **Dados:** Peso 500kg, Altura 3.0m
- **Resultado Esperado:** Cálculo realizado com sucesso

Cenário 3: Entrada Inválida

- **Dados:** Peso -10kg
- **Resultado Esperado:** Mensagem de erro clara

Cenário 4: Precisão Numérica

- **Dados:** Valores que resultam em dízimas periódicas
- **Resultado Esperado:** Arredondamento correto

6. Riscos Identificados

Risco Alto

- **R01:** Cálculo incorreto do IMC afeta diagnóstico médico
- **R02:** Classificação errada pode levar a recomendações inadequadas

Risco Médio

- **R03:** Validações insuficientes permitem entrada de dados irreais
- **R04:** Overflow em cálculos com valores extremos

Risco Baixo

- **R05:** Arredondamento inconsistente entre diferentes execuções
-

Tabela de Decisão - API ViaCEP

Consulta por Endereço: UF/Cidade/Logradouro

Condições de Entrada

- **C1 — UF válida**
A UF informada deve existir no Brasil (ex.: SP, RJ, MG etc.).
- **C2 — Cidade existe**
A cidade deve estar cadastrada para a UF informada.
- **C3 — Acentuação correta**
O nome da cidade deve ser digitado com acentuação correta.
- **C4 — Logradouro existe**
O logradouro deve estar cadastrado dentro da cidade informada.

Ações Esperadas

- **A1 — Retornar dados**
A API retorna uma lista contendo os endereços encontrados.
- **A2 — Retornar vazio**
A API retorna um array vazio ([]).

- **A3 — Retornar erro**
A API retorna um status 400 ou uma mensagem de erro.

Tabela de Decisão

Regra	R1	R2	R3	R4	R5	R6	R7	R8
CONDIÇÕES								
C1: UF válida	✓	✓	✓	✓	✓	✓	✗	✗
C2: Cidade existe	✓	✓	✓	✓	✗	✗	-	-
C3: Acentuação correta	✓	✓	✗	✗	-	-	-	-
C4: Logradouro existe	✓	✗	✓	✗	-	-	-	-

Legenda: ✓ = Sim, ✗ = Não, - = Não aplicável

Descrição das Regras

R1: Tudo Válido ✓

- **Condições:** UF válida + Cidade existe + Acentuação correta + Logradouro existe
- **Exemplo:** /ws/SP/São Paulo/Avenida Paulista/json/
- **Resultado:** Status 200, array com endereços da Av. Paulista
- **Justificativa:** Todas as condições atendidas, sistema retorna dados

R2: Logradouro Inexistente

- **Condições:** UF válida + Cidade existe + Acentuação correta + Logradouro NÃO existe
- **Exemplo:** /ws/SP/São Paulo/Rua Inexistente XYZ 999/json/
- **Resultado:** Status 200, array vazio []

- **Justificativa:** Parâmetros válidos mas logradouro não encontrado

R3: Sem Acentuação, Logradouro Existe

- **Condições:** UF válida + Cidade existe + Sem acentuação + Logradouro existe
- **Exemplo:** /ws/SP/Sao Paulo/Avenida Paulista/json/
- **Resultado:** Status 200, array com dados
- **Justificativa:** API tolera ausência de acentuação

R4: Sem Acentuação, Logradouro Inexistente

- **Condições:** UF válida + Cidade existe + Sem acentuação + Logradouro NÃO existe
- **Exemplo:** /ws/SP/Sao Paulo/Rua Inexistente/json/
- **Resultado:** Status 200, array vazio
- **Justificativa:** Busca válida mas sem resultados

R5: Cidade Inexistente com Acentuação

- **Condições:** UF válida + Cidade NÃO existe + Acentuação correta
- **Exemplo:** /ws/SP/Cidade Inventada/Qualquer Rua/json/
- **Resultado:** Status 400 ou array vazio
- **Justificativa:** Cidade não cadastrada

R6: Cidade Inexistente sem Acentuação

- **Condições:** UF válida + Cidade NÃO existe + Sem acentuação
- **Exemplo:** /ws/SP/Cidade Falsa/Rua Teste/json/
- **Resultado:** Status 400 ou array vazio
- **Justificativa:** Cidade não encontrada

R7: UF Inválida com Acentuação

- **Condições:** UF NÃO válida + Acentuação correta
- **Exemplo:** /ws/XX/São Paulo/Avenida Paulista/json/
- **Resultado:** Status 400
- **Justificativa:** UF não existe no Brasil

R8: UF Inválida sem Acentuação

- **Condições:** UF NÃO válida + Sem acentuação
 - **Exemplo:** /ws/ZZ/Qualquer/Lugar/json/
 - **Resultado:** Status 400
 - **Justificativa:** UF inválida
-

Partições de Equivalência

Para CEP

Partição	Descrição	Valores	Resultado Esperado
PE1	CEP válido e existente	01310100, 20040020	Status 200 + dados
PE2	CEP válido mas inexistente	99999999	Status 200 + erro:true
PE3	CEP com formato inválido	0131010a, ABC12345	Status 400
PE4	CEP com tamanho incorreto	123, 123456789	Status 400
PE5	CEP vazio ou null	"" , null	Status 400

Para UF

Partição	Descrição	Exemplos	Resultado
PE6	UF válida	SP, RJ, MG, RS	Aceito
PE7	UF inválida	XX, ZZ, 12	Erro 400

Para Cidade

Partição	Descrição	Exemplos	Resultado
PE8	Cidade com acentuação	São Paulo, Brasília	Aceito
PE9	Cidade sem acentuação	Sao Paulo, Brasilia	Aceito
PE10	Cidade inexistente	Cidade Falsa	Vazio/Erro

Análise de Valor Limite

CEP (8 dígitos)

- **Mínimo válido:** 00000001
- **Máximo válido:** 99999999
- **Valores de teste:**
 - Limites inferiores: 00000000, 00000001
 - Limites superiores: 99999998, 99999999
 - Tamanho: 7 dígitos (inválido), 8 dígitos (válido), 9 dígitos (inválido)

Nome de Cidade (caracteres)

- **Mínimo:** 1 caractere
 - **Máximo:** ~50 caracteres (limite prático)
 - **Casos especiais:**
 - Cidade com 1 letra: "A" (se existir)
 - Cidade muito longa
 - Caracteres especiais: hífen, apóstrofos
-

Justificativa da Estratégia de Teste

1. Cobertura de Cenários Críticos

A tabela cobre todas as combinações importantes de condições que podem afetar o resultado da API:

- Validação de UF (critério mais restritivo)
- Existência de cidade (segundo critério)
- Tolerância a acentuação (funcionalidade adicional)
- Busca de logradouro (objetivo final)

2. Riscos Abordados

- **Risco Alto:** UF inválida poderia causar erro de sistema → Testado em R7/R8
- **Risco Médio:** Acentuação incorreta poderia impedir buscas válidas → Testado em R3/R4
- **Risco Baixo:** Logradouro inexistente deveria retornar vazio, não erro → Testado em R2/R4

3. Partições Escolhidas

As partições de equivalência foram definidas baseadas em:

- Formato de entrada (CEP com 8 dígitos)
- Existência no banco de dados (CEP cadastrado vs não cadastrado)
- Validação de caracteres (letras vs números)
- Estados brasileiros (27 UFs válidas vs códigos inválidos)

4. Valores Limite

Escolhidos para testar:

- Limites numéricos do CEP (00000000 a 99999999)
- Tamanho da string (7, 8, 9 dígitos)
- Transições entre faixas de classificação